



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №2

«Обчислювальна складність алгоритмів сортування»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Порєв В. М.*

Тема: «Обчислювальна складність алгоритмів сортування».

Мета: Закріплення навичок практичної оцінки алгоритмічної складності логічних алгоритмів на прикладі алгоритмів сортування.

Хід роботи

Мій порядковий номер у списку групи: 6, звідси я маю реалізувати такий алгоритм сортування: Порозрядне сортування, алгоритмічна складність якого $O(nk)$, де k – кількість розрядів (для десяткової системи числення, це максимальна кількість цифр серед чисел), n – розмір масиву (кількість чисел). Наведемо блок-схему алгоритму порозрядного сортування:

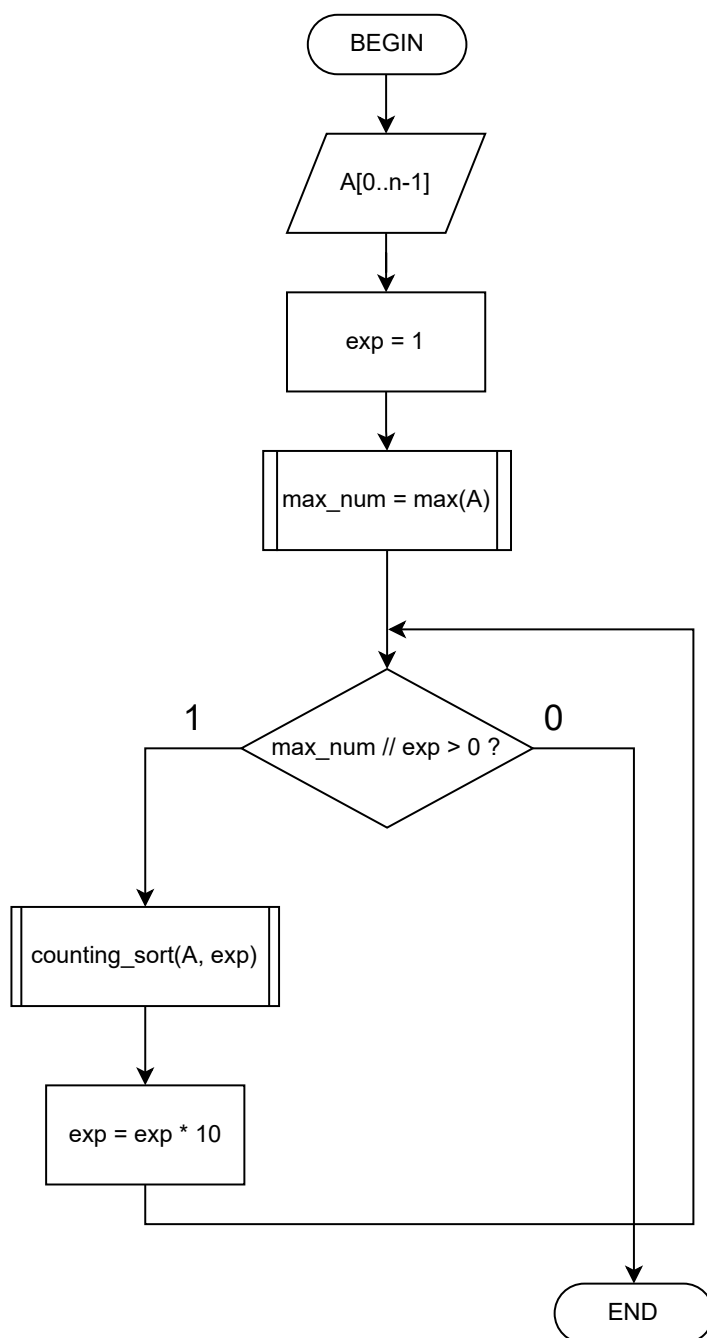


Рис. 1: Блок-схема алгоритму RadixSort

Потрібні до цього алгоритму уточнення та коментарі:

У побудованому алгоритмі використано допоміжні процеси $\text{max}(A)$ та $\text{counting_sort}(A, \text{exp})$, які доцільно окремо пояснити.

Процес $\text{max}(A)$. Операція $\text{max}(A)$ означає пошук максимального елемента масиву A . Цей процес потрібен для того, щоб визначити, скільки розрядів має найбільше число масиву, а отже — скільки проходів порозрядного сортування необхідно виконати.

Наприклад, якщо найбільший елемент масиву дорівнює 802, то він має 3 розряди. Отже, алгоритм має виконати 3 проходи:

1. за розрядом одиниць;
2. за розрядом десятків;
3. за розрядом сотень.

У програмній реалізації після знаходження максимального елемента використовується умова

$$\text{max_num} // \text{exp} > 0,$$

яка перевіряє, чи залишився ще необроблений розряд.

Процес $\text{counting_sort}(A, \text{exp})$. Процес $\text{counting_sort}(A, \text{exp})$ означає сортування підрахунком за поточним розрядом. Це допоміжна процедура, яка впорядковує елементи масиву не за всім числом повністю, а лише за однією цифрою, що визначається значенням exp .

Поточна цифра обчислюється за формулою

$$\text{digit} = (\text{num} // \text{exp}) \% 10.$$

При цьому:

- якщо $\text{exp} = 1$, отримуємо цифру одиниць;
- якщо $\text{exp} = 10$, отримуємо цифру десятків;
- якщо $\text{exp} = 100$, отримуємо цифру сотень.

Суть сортування підрахунком у цьому алгоритмі. Сортування підрахунком виконується у кілька етапів:

1. для кожного елемента масиву визначається цифра поточного розряду;
2. підраховується, скільки разів зустрічається кожне можливе значення цифри від 0 до 9;
3. на основі отриманих підрахунків формується масив позицій;
4. елементи розміщуються у допоміжному масиві у правильному порядку;
5. після цього результат копіюється назад у вихідний масив.

Оскільки в десятковій системі числення можливі лише 10 цифр $(0, 1, \dots, 9)$, для підрахунку використовується допоміжний масив довжини 10.

Чому саме сортування підрахунком використовується в Radix Sort. У порозрядному сортуванні важливо, щоб сортування за кожним окремим розрядом було стабільним, тобто щоб елементи з однаковим значенням поточного розряду зберігали свій попередній відносний порядок. Саме тому сортування підрахунком добре підходить для реалізації Radix Sort.

Для реалізації алгоритмів та GUI середовища я використовуватиму мову програмування Python і інструмент Tkinter та бібліотеку customtkinter.

Програмний код для реалізації алгоритму RadixSort та CountingSort:

(src/linear_radix_sort.py)

```
1 from __future__ import annotations
2
3 def radix_sort(arr: list[int]) -> tuple[list[int], int]:
4     ops = 0
5
6     ops += 1 # перевірка if not arr
7     if not arr:
8         ops += 1 # return
9         return arr, ops
10
11     max_num = max(arr)
12     ops += len(arr) # пошук максимуму = n операцій
13
14     exp = 1
15     ops += 1 # присвоєння
16
17     ops += 1 # перша перевірка while
18     while max_num // exp > 0:
19         ops += 1 # обчислення max_num // exp > 0
20
21         _, sub_ops = counting_sort_by_digit(arr, exp)
22         ops += sub_ops
23
24         exp *= 10
25         ops += 1 # множення + присвоєння
26
27         ops += 1 # наступна перевірка while
28
29     ops += 1 # return
30     return arr, ops
31
32 def counting_sort_by_digit(arr: list[int], exp: int) -> tuple[list[int], int]:
33     ops = 0
34
35     n = len(arr)
36     ops += 1
37
38     output = [0] * n
39     ops += n # створення масиву довжини n
40
41     count = [0] * 10
42     ops += 10 # створення масиву довжини 10
43
```

```

44     # Підрахунок цифр
45     for num in arr:
46         ops += 1 # ітерація циклу
47         digit = (num // exp) % 10
48         ops += 2 # // i %
49         count[digit] += 1
50         ops += 1 # інкремент
51
52     ops += 1 # завершальна перевірка циклу
53
54     # Префіксні суми
55     for i in range(1, 10):
56         ops += 1 # ітерація циклу
57         count[i] += count[i - 1]
58         ops += 2 # індексація + додавання
59
60     ops += 1 # завершальна перевірка циклу
61
62     # Побудова output справа наліво
63     for i in range(n - 1, -1, -1):
64         ops += 1 # ітерація циклу
65         digit = (arr[i] // exp) % 10
66         ops += 3 # arr[i], //, %
67         output[count[digit] - 1] = arr[i]
68         ops += 3 # count[digit], -1, присвоєння
69         count[digit] -= 1
70         ops += 2 # доступ + декремент
71
72     ops += 1 # завершальна перевірка циклу
73
74     # Копіювання назад
75     for i in range(n):
76         ops += 1 # ітерація циклу
77         arr[i] = output[i]
78         ops += 2 # читання + присвоєння
79
80     ops += 1 # завершальна перевірка циклу
81
82     ops += 1 # return
83     return arr, ops

```

Програмний код для побудови графіки фактичної та теоретичної асимптотичної часової складності (src/linear_graph_build.py):

```

1  from __future__ import annotations
2
3  import random
4  from collections.abc import Callable
5
6  from matplotlib.figure import Figure
7
8
9  def build_actual_vs_theoretical_figure(
10     sort_func: Callable[[list[int]], tuple[list[int], int]]

```

```

11 ) -> Figure:
12     sizes: list[int] = []
13     actual_ops: list[int] = []
14     theoretical_ops: list[int] = []
15
16     d = 4
17     k = 10
18
19     for n in range(10, 501, 10):
20         arr = [random.randint(0, 9999) for _ in range(n)]
21         _, ops = sort_func(arr.copy())
22
23         sizes.append(n)
24         actual_ops.append(ops)
25         theoretical_ops.append(d * (n + k))
26
27     scale = actual_ops[-1] / theoretical_ops[-1]
28     theoretical_scaled: list[float] = [x * scale for x in theoretical_ops]
29
30     fig = Figure(figsize=(9, 6), dpi=100)
31     ax = fig.add_subplot(111)
32
33     ax.plot(sizes, actual_ops, marker="o", label="Фактична кількість операцій")
34     ax.plot(
35         sizes,
36         theoretical_scaled,
37         linestyle="--",
38         label="Теоретична залежність d(n+k)",
39     )
40
41     ax.set_xlabel("Розмір масиву n")
42     ax.set_ylabel("Кількість операцій")
43     ax.set_title("Порівняння фактичної та теоретичної залежностей для Radix Sort")
44     ax.grid(True)
45     ax.legend()
46
47     fig.tight_layout()
48     return fig

```

Програмний код основної програми (app.py):

```

1  from __future__ import annotations
2
3  import json
4  import re
5  import tkinter as tk
6  from dataclasses import dataclass
7  from pathlib import Path
8  from tkinter import filedialog
9  from typing import Callable
10
11  import customtkinter as ctk
12  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

```

```

13 from matplotlib.figure import Figure
14
15 from src.radix_sort import radix_sort
16 from src.graph_build import build_actual_vs_theoretical_figure
17
18
19 MAIN_THEME: dict[str, str] = {
20     "bg": "#000000",
21     "topbar": "#000000",
22     "panel": "#090909",
23     "surface": "#111111",
24     "surface_soft": "#1B1B1B",
25     "text": "#F5F7FA",
26     "muted": "#97A1AD",
27     "accent": "#29A8FF",
28     "accent_hover": "#44B4FF",
29     "accent_text": "#06121B",
30     "segment": "#131313",
31     "segment_hover": "#1E1E1E",
32     "segment_active": "#1F2D3C",
33     "error": "#FF5B71",
34 }
35
36 INT_LIST_SPLIT_PATTERN = re.compile(r"[\s]+")
37
38
39 @dataclass(frozen=True)
40 class PageConfig:
41     page_id: str
42     nav_title: str
43     title: str
44     subtitle: str
45
46
47 def parse_int_array(raw: str) -> list[int]:
48     value = raw.strip()
49     if not value:
50         raise ValueError("Поле масиву не може бути порожнім")
51
52     parts = [part for part in INT_LIST_SPLIT_PATTERN.split(value) if part]
53     if not parts:
54         raise ValueError("Не вдалося розпізнати жодного числа")
55
56     try:
57         return [int(part) for part in parts]
58     except ValueError as exc:
59         raise ValueError(
60             "Масив має містити лише цілі числа, розділені пробілами або комами"
61         ) from exc
62
63
64 class BasePage(ctk.CTkFrame):
65     def __init__(
66         self,

```

```

67         master: ctk.CTkFrame,
68         config: PageConfig,
69         fonts: dict[str, ctk.CTkFont],
70     ) -> None:
71         super().__init__(master, corner_radius=0)
72         self.config_data = config
73         self.fonts = fonts
74
75         self.grid_columnconfigure(0, weight=1)
76
77     def apply_theme(self, palette: dict[str, str]) -> None:
78         self.configure(fg_color=palette["bg"])
79
80
81 class SortPage(BasePage):
82     def __init__(
83         self,
84         master: ctk.CTkFrame,
85         config: PageConfig,
86         fonts: dict[str, ctk.CTkFont],
87         sort_func: Callable[[list[int]], tuple[list[int], int]],
88     ) -> None:
89         super().__init__(master, config, fonts)
90         self.sort_func = sort_func
91
92         self.result_var = tk.StringVar(value="Ready")
93         self.error_var = tk.StringVar(value="")
94
95         self._build_header()
96         self._build_form()
97         self._build_actions()
98         self._build_output()
99
100     def _build_header(self) -> None:
101         self.header_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
102         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
103         self.header_card.grid_columnconfigure(0, weight=1)
104
105         self.title_label = ctk.CTkLabel(
106             self.header_card,
107             text=self.config_data.title,
108             anchor="w",
109             font=self.fonts["title"],
110         )
111         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
112
113         self.subtitle_label = ctk.CTkLabel(
114             self.header_card,
115             text=self.config_data.subtitle,
116             anchor="w",
117             font=self.fonts["subtitle"],
118         )
119         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
↵ 12))

```



```

120
121 def _build_form(self) -> None:
122     self.form_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
123     self.form_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
124     self.form_card.grid_columnconfigure(0, weight=1)
125
126     self.array_label = ctk.CTkLabel(
127         self.form_card,
128         text="Array:",
129         anchor="w",
130         font=self.fonts["label"],
131     )
132     self.array_label.grid(row=0, column=0, sticky="w", padx=20, pady=(14, 6))
133
134     self.array_entry = ctk.CTkEntry(
135         self.form_card,
136         height=40,
137         corner_radius=10,
138         font=self.fonts["body"],
139         placeholder_text="Наприклад: 170, 45, 75, 90, 802, 24, 2, 66",
140     )
141     self.array_entry.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 10))
142
143     self.hint_label = ctk.CTkLabel(
144         self.form_card,
145         text="Розділювачі: пробіл або кома.",
146         anchor="w",
147         font=self.fonts["body"],
148     )
149     self.hint_label.grid(row=2, column=0, sticky="w", padx=20, pady=(0, 14))
150
151 def _build_actions(self) -> None:
152     self.actions = ctk.CTkFrame(self, fg_color="transparent")
153     self.actions.grid(row=2, column=0, sticky="ew", pady=(0, 10))
154     self.actions.grid_columnconfigure((0, 1, 2), weight=1)
155
156     self.compute_button = ctk.CTkButton(
157         self.actions,
158         text="Sort",
159         height=40,
160         corner_radius=11,
161         font=self.fonts["button"],
162         command=self.compute,
163     )
164     self.compute_button.grid(row=0, column=0, sticky="ew", padx=(0, 7))
165
166     self.load_button = ctk.CTkButton(
167         self.actions,
168         text="Load JSON",
169         height=40,
170         corner_radius=11,
171         font=self.fonts["button"],
172         command=self.load_json,
173     )

```

```

174         self.load_button.grid(row=0, column=1, sticky="ew", padx=7)
175
176         self.clear_button = ctk.CTkButton(
177             self.actions,
178             text="Clear",
179             height=40,
180             corner_radius=11,
181             font=self.fonts["button"],
182             command=self.clear_all,
183         )
184         self.clear_button.grid(row=0, column=2, sticky="ew", padx=(7, 0))
185
186     def _build_output(self) -> None:
187         self.output_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
188         self.output_card.grid(row=3, column=0, sticky="ew")
189         self.output_card.grid_columnconfigure(0, weight=1)
190
191         self.result_title = ctk.CTkLabel(
192             self.output_card,
193             text="Result",
194             anchor="w",
195             font=self.fonts["label"],
196         )
197         self.result_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 6))
198
199         self.result_label = ctk.CTkLabel(
200             self.output_card,
201             textvariable=self.result_var,
202             anchor="w",
203             justify="left",
204             wraplength=860,
205             font=self.fonts["result"],
206         )
207         self.result_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
208
209         self.error_label = ctk.CTkLabel(
210             self.output_card,
211             textvariable=self.error_var,
212             anchor="w",
213             justify="left",
214             wraplength=860,
215             font=self.fonts["body"],
216         )
217         self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 14))
218
219     def load_json(self) -> None:
220         path = filedialog.askopenfilename(
221             title="Select JSON file",
222             filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
223         )
224         if not path:
225             return
226
227         try:

```

```

228     raw_data = Path(path).read_text(encoding="utf-8")
229     loaded = json.loads(raw_data)
230
231     if not isinstance(loaded, dict):
232         raise ValueError("JSON root must be an object")
233
234     if "array" not in loaded:
235         raise ValueError("JSON must contain key 'array'")
236
237     array_data = loaded["array"]
238
239     if not isinstance(array_data, list):
240         raise TypeError("'array' must be a list")
241
242     if not all(isinstance(x, int) for x in array_data):
243         raise TypeError("'array' must contain only integers")
244
245     prepared = ", ".join(str(x) for x in array_data)
246
247     self.array_entry.delete(0, tk.END)
248     self.array_entry.insert(0, prepared)
249
250     self.result_var.set("JSON loaded. Press Sort.")
251     self.error_var.set("")
252 except Exception as exc:
253     self.result_var.set("No result")
254     self.error_var.set(f"Error: {exc}")
255
256 def compute(self) -> None:
257     try:
258         raw = self.array_entry.get()
259         arr = parse_int_array(raw)
260         sorted_arr, ops = self.sort_func(arr.copy())
261
262         self.result_var.set(f"Sorted array: {sorted_arr}\nOperations: {ops}")
263         self.error_var.set("")
264     except Exception as exc:
265         self.result_var.set("No result")
266         self.error_var.set(f"Error: {exc}")
267
268 def clear_all(self) -> None:
269     self.array_entry.delete(0, tk.END)
270     self.result_var.set("Ready")
271     self.error_var.set("")
272
273 def apply_theme(self, palette: dict[str, str]) -> None:
274     super().apply_theme(palette)
275
276     for card in (self.header_card, self.form_card, self.output_card):
277         card.configure(
278             fg_color=palette["panel"],
279             border_color=palette["surface_soft"],
280         )
281

```

```

282     self.title_label.configure(text_color=palette["text"])
283     self.subtitle_label.configure(text_color=palette["muted"])
284     self.array_label.configure(text_color=palette["muted"])
285     self.hint_label.configure(text_color=palette["muted"])
286
287     self.array_entry.configure(
288         fg_color=palette["surface"],
289         border_color=palette["surface_soft"],
290         text_color=palette["text"],
291         placeholder_text_color=palette["muted"],
292     )
293
294     self.compute_button.configure(
295         fg_color=palette["accent"],
296         hover_color=palette["accent_hover"],
297         text_color=palette["accent_text"],
298     )
299     self.load_button.configure(
300         fg_color=palette["segment"],
301         hover_color=palette["segment_hover"],
302         text_color=palette["text"],
303     )
304     self.clear_button.configure(
305         fg_color=palette["segment"],
306         hover_color=palette["segment_hover"],
307         text_color=palette["text"],
308     )
309
310     self.result_title.configure(text_color=palette["muted"])
311     self.result_label.configure(text_color=palette["text"])
312     self.error_label.configure(text_color=palette["error"])
313
314
315 class GraphPage(BasePage):
316     def __init__(
317         self,
318         master: ctk.CTkFrame,
319         config: PageConfig,
320         fonts: dict[str, ctk.CTkFont],
321         graph_func: Callable[[Callable[[list[int]], tuple[list[int], int]]],
322                               ↪ Figure],
323         sort_func: Callable[[list[int]], tuple[list[int], int]],
324     ) -> None:
325         super().__init__(master, config, fonts)
326         self.graph_func = graph_func
327         self.sort_func = sort_func
328
329         self.status_var = tk.StringVar(value="Ready to build graph")
330         self.error_var = tk.StringVar(value="")
331         self.canvas: FigureCanvasTkAgg | None = None
332
333         self.grid_rowconfigure(3, weight=1)
334
335         self._build_header()

```

```

335     self._build_controls()
336     self._build_output()
337     self._build_plot_area()
338
339     def _build_header(self) -> None:
340         self.header_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
341         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
342         self.header_card.grid_columnconfigure(0, weight=1)
343
344         self.title_label = ctk.CTkLabel(
345             self.header_card,
346             text=self.config_data.title,
347             anchor="w",
348             font=self.fonts["title"],
349         )
350         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
351
352         self.subtitle_label = ctk.CTkLabel(
353             self.header_card,
354             text=self.config_data.subtitle,
355             anchor="w",
356             font=self.fonts["subtitle"],
357         )
358         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
359             ↪ 12))
360
361     def _build_controls(self) -> None:
362         self.controls_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
363         self.controls_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
364         self.controls_card.grid_columnconfigure(0, weight=1)
365
366         self.info_label = ctk.CTkLabel(
367             self.controls_card,
368             text=(
369                 "Побудова графіка фактичної кількості операцій "
370                 "та теоретичної залежності Radix Sort."
371             ),
372             anchor="w",
373             justify="left",
374             wraplength=860,
375             font=self.fonts["body"],
376         )
377         self.info_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 12))
378
379         self.build_button = ctk.CTkButton(
380             self.controls_card,
381             text="Build Graph",
382             height=40,
383             corner_radius=11,
384             font=self.fonts["button"],
385             command=self.build_graph,
386         )
387         self.build_button.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 14))

```

```

388 def _build_output(self) -> None:
389     self.output_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
390     self.output_card.grid(row=2, column=0, sticky="ew")
391     self.output_card.grid_columnconfigure(0, weight=1)
392
393     self.status_title = ctk.CTkLabel(
394         self.output_card,
395         text="Status",
396         anchor="w",
397         font=self.fonts["label"],
398     )
399     self.status_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 6))
400
401     self.status_label = ctk.CTkLabel(
402         self.output_card,
403         textvariable=self.status_var,
404         anchor="w",
405         justify="left",
406         wraplength=860,
407         font=self.fonts["result"],
408     )
409     self.status_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
410
411     self.error_label = ctk.CTkLabel(
412         self.output_card,
413         textvariable=self.error_var,
414         anchor="w",
415         justify="left",
416         wraplength=860,
417         font=self.fonts["body"],
418     )
419     self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 14))
420
421 def _build_plot_area(self) -> None:
422     self.plot_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
423     self.plot_card.grid(row=3, column=0, sticky="nsew", pady=(14, 0))
424     self.plot_card.grid_columnconfigure(0, weight=1)
425     self.plot_card.grid_rowconfigure(0, weight=1)
426
427     self.plot_container = ctk.CTkFrame(self.plot_card, fg_color="transparent")
428     self.plot_container.grid(row=0, column=0, sticky="nsew", padx=12, pady=12)
429     self.plot_container.grid_columnconfigure(0, weight=1)
430     self.plot_container.grid_rowconfigure(0, weight=1)
431
432 def build_graph(self) -> None:
433     try:
434         self.status_var.set("Building graph...")
435         self.error_var.set("")
436         self.update_idletasks()
437
438         fig = self.graph_func(self.sort_func)
439
440         if self.canvas is not None:
441             self.canvas.get_tk_widget().destroy()

```

```

442         self.canvas = FigureCanvasTkAgg(fig, master=self.plot_container)
443         self.canvas.draw()
444         self.canvas.get_tk_widget().grid(row=0, column=0, sticky="nsew")
445
446         self.status_var.set("Graph was built successfully")
447     except Exception as exc:
448         self.status_var.set("Graph was not built")
449         self.error_var.set(f"Error: {exc}")
450
451
452 def apply_theme(self, palette: dict[str, str]) -> None:
453     super().apply_theme(palette)
454
455     for card in (self.header_card, self.controls_card, self.output_card,
456 ↪ self.plot_card):
457         card.configure(
458             fg_color=palette["panel"],
459             border_color=palette["surface_soft"],
460         )
461
462     self.title_label.configure(text_color=palette["text"])
463     self.subtitle_label.configure(text_color=palette["muted"])
464     self.info_label.configure(text_color=palette["muted"])
465
466     self.build_button.configure(
467         fg_color=palette["accent"],
468         hover_color=palette["accent_hover"],
469         text_color=palette["accent_text"],
470     )
471
472     self.status_title.configure(text_color=palette["muted"])
473     self.status_label.configure(text_color=palette["text"])
474     self.error_label.configure(text_color=palette["error"])
475
476 class AlgorithmsApp(ctlk.CTk):
477     def __init__(self) -> None:
478         super().__init__()
479         ctlk.set_appearance_mode("dark")
480
481         self.title("RadixSortApp")
482         self.resizable(False, False)
483         self.geometry("720x900")
484
485         self.current_page = "sort"
486         self.nav_buttons: dict[str, ctlk.CTkButton] = {}
487         self.pages: dict[str, BasePage] = {}
488
489         self.fonts: dict[str, ctlk.CTkFont] = {
490             "title": ctlk.CTkFont(family="Segoe UI", size=27, weight="bold"),
491             "subtitle": ctlk.CTkFont(family="Segoe UI", size=13),
492             "label": ctlk.CTkFont(family="Segoe UI", size=13, weight="bold"),
493             "body": ctlk.CTkFont(family="Segoe UI", size=12),
494             "button": ctlk.CTkFont(family="Segoe UI", size=12, weight="bold"),

```

```

495         "result": ctk.CTkFont(family="Consolas", size=16, weight="bold"),
496         "nav": ctk.CTkFont(family="Segoe UI", size=12, weight="bold"),
497     }
498
499     self.page_configs: tuple[PageConfig, ...] = (
500         PageConfig(
501             page_id="sort",
502             nav_title="Sort",
503             title="Radix Sort",
504             subtitle="Сортування масиву цілих чисел та підрахунок кількості
                    ↪ операцій",
505         ),
506         PageConfig(
507             page_id="graph",
508             nav_title="Graph",
509             title="Radix Sort Graph",
510             subtitle="Порівняння фактичної та теоретичної залежностей",
511         ),
512     )
513
514     self.grid_columnconfigure(0, weight=1)
515     self.grid_rowconfigure(1, weight=1)
516
517     self._build_topbar()
518     self._build_pages()
519     self._apply_theme()
520     self.show_page(self.current_page)
521
522     def _build_topbar(self) -> None:
523         self.topbar = ctk.CTkFrame(self, corner_radius=0)
524         self.topbar.grid(row=0, column=0, sticky="ew")
525         self.topbar.grid_columnconfigure(0, weight=1)
526
527         self.nav_frame = ctk.CTkFrame(self.topbar, fg_color="transparent")
528         self.nav_frame.grid(row=0, column=0, sticky="w", padx=20, pady=(18, 12))
529
530         for index, page in enumerate(self.page_configs):
531             button = ctk.CTkButton(
532                 self.nav_frame,
533                 text=page.nav_title,
534                 width=110,
535                 height=36,
536                 corner_radius=10,
537                 font=self.fonts["nav"],
538                 border_width=1,
539                 command=lambda pid=page.page_id: self.show_page(pid),
540             )
541             button.grid(row=0, column=index, padx=(0, 8))
542             self.nav_buttons[page.page_id] = button
543
544     def _build_pages(self) -> None:
545         self.page_host = ctk.CTkFrame(self, corner_radius=0)
546         self.page_host.grid(row=1, column=0, sticky="nsew", padx=20, pady=(0, 20))
547         self.page_host.grid_rowconfigure(0, weight=1)

```



```

548         self.page_host.grid_columnconfigure(0, weight=1)
549
550         sort_page = SortPage(
551             self.page_host,
552             self.page_configs[0],
553             self.fonts,
554             radix_sort,
555         )
556         sort_page.grid(row=0, column=0, sticky="nsew")
557         self.pages["sort"] = sort_page
558
559         graph_page = GraphPage(
560             self.page_host,
561             self.page_configs[1],
562             self.fonts,
563             build_actual_vs_theoretical_figure,
564             radix_sort,
565         )
566         graph_page.grid(row=0, column=0, sticky="nsew")
567         self.pages["graph"] = graph_page
568
569     def show_page(self, page_id: str) -> None:
570         self.current_page = page_id
571         self.pages[page_id].tkraise()
572         self._refresh_nav_buttons()
573
574     def _refresh_nav_buttons(self) -> None:
575         palette = MAIN_THEME
576         for page_id, button in self.nav_buttons.items():
577             if page_id == self.current_page:
578                 button.configure(
579                     fg_color=palette["segment_active"],
580                     hover_color=palette["segment_active"],
581                     text_color=palette["text"],
582                     border_color=palette["segment_active"],
583                 )
584             else:
585                 button.configure(
586                     fg_color=palette["segment"],
587                     hover_color=palette["segment_hover"],
588                     text_color=palette["muted"],
589                     border_color=palette["surface_soft"],
590                 )
591
592     def _apply_theme(self) -> None:
593         palette = MAIN_THEME
594
595         self.configure(fg_color=palette["bg"])
596         self.topbar.configure(fg_color=palette["topbar"])
597         self.page_host.configure(fg_color=palette["bg"])
598
599         for page in self.pages.values():
600             page.apply_theme(palette)
601

```

```

602         self._refresh_nav_buttons()
603
604
605 def main() -> None:
606     app = AlgorithmsApp()
607     app.mainloop()
608
609
610 if __name__ == "__main__":
611     main()

```

Короткий опис функціоналу основної програми:

- GUI побудовано на `customtkinter` у мінімалістичному стилі з темною кольоровою схемою.
- Програма містить 2 основні сторінки: `Sort` та `Graph`.
- Сторінка `Sort` призначена для введення масиву цілих чисел, виконання порозрядного сортування та виведення відсортованого масиву разом із кількістю виконаних операцій.
- Ввід масиву підтримується як вручну через поле введення, так і шляхом завантаження JSON-файлу.
- Для сторінки `Sort` передбачено кнопки `Sort`, `Load JSON` та `Clear`.
- JSON завантажується з файлу, при цьому очікується структура вигляду `{"array": [170, 45, 75, 90, 802, 24, 2, 66]}`.
- Під час завантаження JSON виконується перевірка, що кореневий елемент є об'єктом, що існує ключ `array`, що його значення є списком, а всі елементи списку є цілими числами.
- Сторінка `Graph` призначена для побудови графіка фактичної кількості операцій та теоретичної залежності алгоритму порозрядного сортування.
- Графік не відкривається в окремому вікні, а вбудовується безпосередньо в інтерфейс програми за допомогою `FigureCanvasTkAgg`.
- Для сторінки `Graph` передбачено кнопку `Build Graph`, яка викликає побудову графіка на основі функції сортування.
- У програмі реалізовано окремі блоки для відображення результату, статусу виконання та повідомлень про помилки.
- Помилки не зупиняють виконання програми, а відображаються в інтерфейсі у форматі `Error: ....`
- Навігація між сторінками виконується через верхню панель перемикачів вкладок.

Скріншоти демонстрації програми:

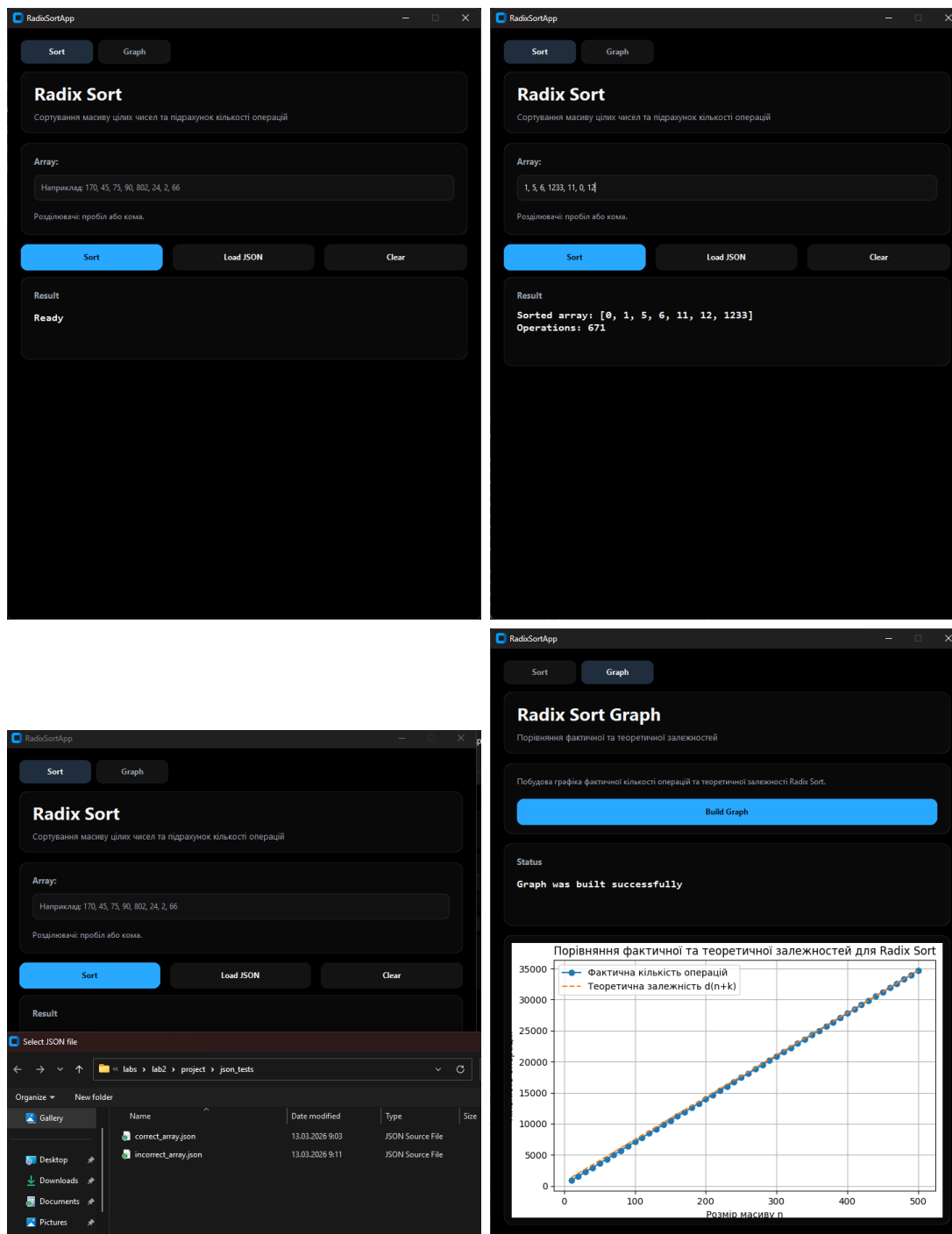


Рис. 2: Скріншоти демонстрації програми

Аналіз результатів

У ході виконання роботи було реалізовано алгоритм порозрядного сортування (Radix Sort) для масиву цілих невід'ємних чисел, а також програмний модуль для побудови графічної залежності фактичної кількості операцій від розміру вхідного масиву та її порівняння з теоретичною оцінкою.

У результаті тестування встановлено, що алгоритм коректно впорядковує елементи масиву за зростанням. Робота алгоритму ґрунтується на послідовному сортуванні елементів за окремими розрядами числа, починаючи з молодшого розряду і закінчуючи старшим.

Для кожного розряду використовується стабільне сортування підрахунком, що забезпечує збереження відносного порядку елементів із однаковими значеннями поточної цифри.

Під час експериментального дослідження було виконано підрахунок кількості операцій, які здійснює алгоритм при сортуванні масивів різної довжини. На основі отриманих значень побудовано графік фактичної кількості операцій. Також було побудовано теоретичну залежність, що відповідає часовій складності алгоритму порозрядного сортування.

Аналіз графіка показує, що зі збільшенням кількості елементів масиву кількість операцій зростає майже лінійно. Це узгоджується з теоретичною оцінкою складності алгоритму. Для порозрядного сортування часова складність часто подається у вигляді

$$O(nk),$$

де n — кількість елементів масиву, а k — кількість розрядів у найбільшому числі. У більш детальному вигляді цю складність можна записати як

$$O(k(n + b)),$$

де k — кількість розрядів, а b — основа системи числення. У даній реалізації використовується десяткова система числення, тому $b = 10$ є сталою, а отже складність можна розглядати як лінійну за n при фіксованому числі розрядів.

Крім часової складності, було проаналізовано й витрати пам'яті. Алгоритм використовує допоміжний масив `output` розміру n та масив підрахунків `count` розміру 10, тому просторова складність становить

$$O(n + 10) = O(n).$$

Отримані результати підтверджують, що порозрядне сортування є ефективним для сортування цілих чисел, особливо у випадках, коли кількість розрядів у числах є відносно невеликою. Побудований графік демонструє відповідність між практичними результатами підрахунку операцій та теоретичною оцінкою складності, що підтверджує коректність реалізації алгоритму та правильність виконаного аналізу.